# Chapter 1. Getting Started: Compiling and Running Java

## 1.0 Introduction

This chapter covers some entry-level tasks that you need to know how to do before you can go on. It is said you must crawl before you can walk, and walk before you can ride a bicycle. Before you can try out anything in this book, you need to be able to compile and run your Java code. I start by demonstrating several methods: using the Java Development Kit (JDK), an Integrated Development Environment (IDE), and build tools like Maven and Gradle. Another issue people run into is setting `CLASSPATH` correctly, so that's dealt with next. Deprecation warnings follow after that, because you're likely to encounter them in maintaining old Java code. The chapter ends with some general information about unit testing and assertions.

If you don't have Java installed, you'll need either to download the JDK on its own, particularly to the command-line Java, or download an IDE that includes its own JDK. Be aware that in historical releases there were different downloads, the Java Runtime Environment (JRE) and the JDK. Up until Java 8, the JRE was a smaller download for end users (i.e., just the parts needed to run a Java app, but not to compile one). Since there is less desktop Java than there once was, and since hard disks have gotten much bigger, the JRE was eliminated in favor of creating `jlink` as part of the JDK, to create custom downloads of the JDK (see Recipe 2.17). The JDK download (sometimes referred to as the Java Software Development Kit or SDK) is the full development environment, which you'll want if you're going to develop Java software.

Standard downloads for the current release of Java are available at Oracle's website and from several other sites (not a complete list):

- Eclipse Software Foundation
- Microsoft
- Azul
- Java.net

The entire JDK is maintained as an open source project, and the OpenJDK source tree is used (with changes and additions) to build the commercial and supported Oracle JDKs. The OpenJDK repository has the source code for the entire JDK. You can in fact build your own JDK fairly easily from this GitHub repository; see my article on the JDK source code, about halfway down the page.

If you are going to have more than one version of the JDK on your Unix/Linux/macOS computer, you might want to investigate SDKMAN, which lets you easily install and switch between them.

If you're already happy with your IDE, you may wish to skip some or all of the next few recipes. The coverage is here to ensure that everybody can compile and debug their programs before we move on.

## 1.1 Hello, World: Compiling and Running Java with the Standard JDK

### Problem

You need to compile and run your Java program.

### Solution

This is one of the few areas where your computer's operating system impinges on Java's portability, so let's get these issues out of the way first.

### Discussion

Using the command-line JDK may be the best way to keep up with the very latest improvements in Java. Assuming you have the standard JDK installed in the standard location and/or have set its location in your `PATH`, you should be able to run the command-line JDK tools. Use the commands `javac` to compile and `java` to run your program (and, on Windows only, `javaw` to run a program without a console window), like this:

```
C:\> javac HelloWorld.java

C:\> java HelloWorld
Hello, World

C:\>
```

If the program refers to other classes for which the source is available (in the same directory) and a compiled *.class* file is not, `javac` will automatically compile it for you. Effective with Java 11, for simple programs that don't need any such co-compilation, you can combine the two operations—compiling and executing—simply by passing the Java source file to the `java` command:

```
$ java HelloWorld.java
Hello, Java
$
```

22 Effective with Java 22, the compile-and-run form of the `java` command will find and compile additional source files. Try this on Java 22+:

```
$ cd main/src/main/java/starting
$ ls ???.java
One.java  Two.java
$ java One.java
```

Don't be confused by the compiler's (lack of) output in these examples. Both `javac` and `java` work on the Unix "no news is good news" philosophy: if a program was able to do what you asked it to, it shouldn't bother chattering at you to say that it did so. In the words of W.H. Auden, "had anything been wrong, we should certainly have heard."

There is an optional setting called `CLASSPATH`, discussed in Recipe 1.6, that controls where Java looks for classes. `CLASSPATH`, if set, is used by both `javac` and `java`. In older versions of Java, you had to set your `CLASSPATH` to include "." even to run a simple program from the current directory; this is no longer true on current Java.

Sun/Oracle's `javac` compiler is the official reference implementation. There were historically several alternative open source command-line compilers, including Jikes and Kaffe, but they are, for the most part, no longer actively maintained.

There have also been some Java runtime clones, including Apache Harmony, Japhar, the IBM Jikes Runtime, and even JNode, a complete, standalone operating system written in Java. Since the Sun/Oracle JVM has been released as open source software as OpenJDK under the GNU Public License (GPL), most of these projects are no longer maintained. Harmony, for example, was retired by Apache in 2011. Today, a dozen or so organizations supply builds of OpenJDK for free download (and use). Some are enhanced, including Oracle's commercial version and Azul Platform Prime, a heavily optimized derivative of OpenJDK. One alternative implementation that is still maintained is IBM's J9, (a reimplementation, descended from a runtime that existed before Java, and now hosted at *Eclipse.org*).

## 1.2 Hello, World of Classless Main 21P

### Problem

It is tedious to type out `public class Foo {` and `public static void main(String[] args)` every time you start a new program.

## Solution

Use the preview implicitly declared class and instance main features.

## Discussion

*Implicitly declared class* refers to a main program declared by itself, with no enclosing *class* structure. *Instance main* refers to a non-static `main()` method (with or without a `String[]` argument); such methods are runnable from the command line or an up-to-date IDE, just like traditional `main()` methods. For convenience, I'll lump these together under the unofficial term *classless main*.

A simple main program can now be constructed with as little as:

```
void main() {
    System.out.println("Hello, world");
}
```

It may strike joy or fear into the hearts of long-time Java developers and educators (and book authors), but the preceding is a complete compilation unit. Assume it's stored in a file called *hello.java*. Due to the joys of the source-code launcher (another recent Java feature), we can compile and run this program using just:

```
$ java hello.java
```

At least, we'll be able to when the implicit class and instance main features come out of preview. For now we must type:

```
$ java --enable-preview --source 22 hello.java
```

That's almost more typing than the original form of the Hello program! Fortunately, if you're doing this command-line style on Unix, Linux, macOS, or on Windows with the `git-bash` add-on, you can save this as an alias in your shell startup file. I named it `javapvr` for "java (with preview) run," as this alias cannot be used to run compiled *.class* files; the `--source` is interpreted as meaning that you want to compile and run a *.java* file:

```
alias javapvr='java --enable-preview --source 22'
```

If running in an IDE or build tool, the same options must be applied. In build tools, apply both options to the compilation (but only the `--enable-preview` to the run step). IntelliJ IDEA, for example, when it detects a preview feature, just asks you if it's OK to enable Preview for the given IDE project.

Then you can "live in the future" and just use the `javapvr` command to run *.java* files. And when the future arrives—these two features come out of preview—you can drop the alias and just use the plain `java` command.

Note that any compile or run of a program using preview features will generate a few lines of warnings to remind you that it is a preview. I have removed these warnings from many of the example outputs, to focus on the code and its output.

This implicitly declared class and instance main mechanism is intended as an on-ramp to those getting started with Java. As such, it has a few other differences from "the way we were":

- An implicit class cannot have a package declaration, but may of course have imports.
- An implicit class does not need to have a capitalized filename(!); the generated class name will be taken from the filename (see Example 1-1).
- An instance main does not need to declare its `String[]` argument, but may if it needs access to the command-line arguments.
- An instance main is not static and can call nonstatic methods.

All of this is designed to make it easier to get started with Java; educators no longer need to handwave or explain away all the traditional boilerplate code before beginners can get started writing Java code. *This is huge!*

The longer version shown in Example 1-1 shows more of the pieces of classless main. Again, this is the complete compilation unit.

**Example 1-1. main/src/main/java/starting/HelloClasslessWithMembers.java**

```
import java.lang.reflect.*;  // Unused, just to show import is allowed

int x;          // instance fields allowed

void main(String[] args) {  // instance main
  System.out.println("Hello, world");
  process(1);
}
void process(int n) {    // instance method
  System.out.println("Hello " + n);
}
```

This prints out a couple of Hello messages:

```
C:\> java --enable-preview --source 21 HelloClasslessWithMembers.java
Hello, world
Hello 1
```

We can still compile this like any other Java program. As it's still in preview, we need to add the two command-line arguments used earlier into the compile command. The generated class file can be examined with the standard JDK tool `javap`, and run with the standard `java` command (with the `--enable-preview` argument):

```
$ javac --enable-preview --source 21 HelloClasslessWithMembers.java
$ javap HelloClasslessWithMembers
Compiled from "HelloClasslessWithMembers.java"
final class HelloClasslessWithMembers {
  int x;
  HelloClasslessWithMembers();
  void main(java.lang.String[]);
  void process(int);
}
$ java --enable-preview HelloClasslessWithMembers
Hello, world
Hello 1
```

Note that neither `main()` nor `process()` is static; this is a feature of instance main. These features are described in JEP 445: "Unnamed Classes and Instance Main Methods (Preview)".[1] These two features are both in preview as of Java 22; I hope they will become mainstream as quickly as possible, as it will make it so much easier to teach Java in the future!

**23P** In Java 23, the updated version of this preview feature allows even more simplification, the omission of `System.out` in calls to the ubiquitous `println()`:

```
void main() {
    println("Hello world");
}
```

The elimination of the need to explain `System.out` to Java beginners is another forward step along the road to making Java easier to learn. It's a step that I and others had petitioned the Java team to include (after initially rejecting the idea, they came up with a better, cleaner way of implementing it). The easier-to-use `println()` works by having the compiler auto-import statically the methods from the new class `java.io.IO`, which simply delegates `println()` and `print()` to `System.out`, and `readline()` to the `Console` class described in Recipe 10.8. Again, this is a preview feature and therefore subject to change.

Further, the Java 23 preview provides on-demand auto-importing of everything from the `java.base` module, that is, much of the standard API.

### See Also

Implicitly declared classes, instance main, and `java.io.IO` are discussed with their rationale in the JEP 477 for Java 23 (but only for implicitly declared classes)..

## 1.3 Downloading and Using the Code Examples

### Problem

You want to try out my example code and/or use my utility classes.

### Solution

Download the latest archive of the book source files, unpack it, and run Maven (see Recipe 2.4) to compile the files.

### Discussion

The source code used as examples in this book is included in a couple of source code repositories that have been in continuous development since 1995. These are listed in Table 1-1.

Table 1-1. The main source code repositories

| Repository name | GitHub URL | Package description | Approx. size |
|---|---|---|---|
| *javasrc* | *https://github.com/IanDarwin/javasrc* | Java code examples/demos | 1,470 classes |
| *darwinsys-api* | *https://github.com/IanDarwin/darwinsys-api* | A published API | 200 classes |

You can download these repositories from the GitHub URLs shown in Table 1-1. GitHub allows you to download a ZIP file of the entire repository's current state, as well as view individual files on the web interface. Downloading with `git clone` instead of as an archive is preferred because you can then obtain all my updates at any time with a simple `git pull` command. And with the amount of updating this codebase has undergone for the current release of Java, you are sure to find changes after the book is published.

If you are not familiar with Git, see "CVS, Subversion, Git, Oh My!".

#### javasrc

This is the largest repo and consists primarily of code written to show a particular feature or API. The files are organized into subdirectories by topic, many of which correspond more or less to book chapters—for example, a directory for *strings* examples (Chapter 3), *regex* for regular expressions (Chapter 4), *numbers* (Chapter 5), and so on. The archive also contains the index by name and index by chapter files from the download site, so you can easily find the files you need.

The javasrc library is further broken down into a dozen Maven modules (shown in Table 1-2) so that you don't need all the dependencies for everything on your `CLASSPATH` all the time.

Table 1-2. javasrc Maven modules

| Directory/module name | Description | Reference |
|---|---|---|
| *pom.xml* | Maven parent POM | Recipe 2.4 |
| *desktop* | AWT and Swing stuff (no longer covered in *Java Cookbook*) | *https://darwinsys.com/java/cookbookcuttings* |
| *ee* | Enterprise stuff (no longer covered in *Java Cookbook*) | *https://darwinsys.com/java/cookbookcuttings* |
| *graal* | GraalVM demos | Recipe 1.12 |
| *jlink* | `jlink` demos | Recipe 2.17 |
| *json* | JSON processing | Chapter 16 |
| *main* | Contains the majority of the files, i.e., those not required to be in one of the other modules due to `CLASSPATH` or other issues | *passim* |
| *Rdemo-web* | R demo using a web framework | Recipe 12.2 |
| *restdemo* | REST service demo | Recipe 14.1 |
| *spark* | Apache Spark demo | Recipe 12.1 |
| *testing* | Code for testing | Recipe 2.9 |
| *unsafe* | Demo of `Unsafe` class | "Probably?" |
| *xml* | XML processing (no longer covered in *Java Cookbook*) | *https://darwinsys.com/java/cookbookcuttings* |

### darwinsys-api

I have built up a collection of useful stuff partly by moving some reusable classes from *javasrc* into my own API, which I use in my Java projects. I use example code from it in this book, and I import classes from it into numerous examples. If you want to use these classes in your own project, the easiest way is to add it to your *pom.xml* (or the corresponding information in `build.gradle`):

```
<dependency>
  <groupId>com.darwinsys</groupId>
  <artifactId>darwinsys-api</artifactId>
  <version>1.8.0</version>
</dependency>
```

The version number will change over time. You can search on Maven Central to see the latest, and to see the specification in the format for other build tools.

This API consists of about two dozen com.darwinsys packages, listed in Table 1-3. The structure vaguely parallels the standard Java API; this is intentional. These packages now include around 200 classes and interfaces. Most of them have Javadoc documentation that can be viewed with the source download.

Table 1-3. The com.darwinsys packages

| Package name | Package description |
|---|---|
| com.darwinsys.calendar | Calendar/scheduling |
| com.darwinsys.csv | Classes for comma-separated values files |
| com.darwinsys.database | Classes for dealing with databases in a general way |
| com.darwinsys.diff | Comparison utilities |
| com.darwinsys.formatting | Various formatting helpers |
| com.darwinsys.genericui | Generic GUI stuff |
| com.darwinsys.geo | Classes relating to country codes, provinces/states, and so on |
| com.darwinsys.graphics | Graphics |
| com.darwinsys.html | Classes (only one so far) for dealing with HTML |
| com.darwinsys.io | Classes for input and output operations, using Java's underlying I/O classes |
| com.darwinsys.jclipboard | The `JClipBoard` class |
| com.darwinsys.lang | Classes for dealing with standard features of Java |
| com.darwinsys.locks | Pessimistic locking API |
| com.darwinsys.mail | Classes for dealing with email, mainly a convenience class for sending mail |
| com.darwinsys.model | Sample data models |
| com.darwinsys.net | Networking |
| com.darwinsys.notepad | A very simple "notepad" style implemented in Java |
| com.darwinsys.preso | Presentations |
| com.darwinsys.reflection | Reflection |
| com.darwinsys.regex | Regular expression stuff: an REDemo program, a `grep` variant |
| com.darwinsys.security | Security |
| com.darwinsys.sql | Classes for dealing with SQL databases |
| com.darwinsys.swingui | Classes for helping construct and use Swing GUIs |
| com.darwinsys.swingui.layout | A few interesting LayoutManager implementations |
| com.darwinsys.tel | Telephony |
| com.darwinsys.testdata | Test data generators |
| com.darwinsys.tools | Miscellaneous tools |
| com.darwinsys.unix | Unix helpers |
| com.darwinsys.util | A few miscellaneous utility classes |
| com.darwinsys.workflow | BreakNagger, a tool to interrupt your workflow for your own good |
| com.darwinsys.xml | XML utilities |

Many classes from these packages are used as examples in this book. You'll also find that some of the other examples have imports from the com.darwinsys packages.

### General notes

Your best bet is to use `git clone` to download a copy of both Git projects and then do a `git pull` every few months to get updates. Alternatively, on the book's catalog page you can download a single intersection subset of both libraries that is made up almost ex-

clusively of files actually used in the book. This archive is made from the sources that are dynamically included in the book at formatting time, so it should reflect exactly the examples you see in the book. But it will not include as many examples as the three individual archives, nor is it guaranteed that everything will compile because of missing dependencies, nor will it get updates. But if all you want is to copy pieces into a project you're working on, this may be the one to get. You can find links to all of these files from my own website for this book; just follow the Downloads link.

The two separate repositories contain multiple self-contained projects with support for building with both Eclipse (Recipe 1.4) and Maven (Recipe 2.4). Note that Maven will automatically fetch a vast array of prerequisite libraries when first invoked on a given project, so be sure you're online on a high-speed internet link. Maven will thus ensure that all prerequisites are installed before building. If you choose to build pieces individually, look in the *pom.xml* file for the list of dependencies. Unfortunately, I will not be able to help you with the control files included in the download if you are using tooling other than Eclipse or Maven.

If you have a version of Java older than Java 21, a few files will not compile. While it's better if you upgrade to a newer Java, if that's not possible, then you can make up exclusion elements in Maven or other tooling to avoid compiling these files.

All my code in the two projects is released under the least-restrictive credit-only license, the two-clause BSD license. If you find it useful, incorporate it into your own software. There is no need to write to ask me for permission; just use it, with credit. If you get rich off it, send me some money.

---

TIP

Most of the command-line examples refer to source files, assuming you are in *src/main/java*, and runnable classes, assuming you are in (or have added to your `CLASSPATH`) the build directory (e.g., usually *target/classes*). This will not be mentioned with each example, as doing so would consume a lot of paper.

---

### Caveat lector

The repos have been in development since 1995. This means that you will likely find some code that is not up to date or that no longer reflects best practices. This is not surprising: any body of code will grow old if any part of it is not actively maintained. (Thus, at this point, I invoke Culture Club's song "Do You Really Want to Hurt Me": "Give me time to realize my crime.") Where advice in the book disagrees with some code you found in the repo, keep this in mind. One practice of Extreme Programming is continuous refactoring, the ability to improve any part of the codebase at any time. Don't be surprised if the code in the online source directory differs from what appears in the book; it is a rare month that I don't make some improvement to the code, and the results are committed and pushed quite often. So if there are differences between what's printed in the book and what you get from GitHub, be glad, not sad, for you'll have received the benefit of hindsight. Also, people can contribute easily on GitHub via pull requests; that's what makes it interesting. If you find a bug or an improvement, do send me a pull request! The consolidated archive on the book's catalog page will not be updated as frequently.

#### CVS, SUBVERSION, GIT, OH MY!

Many distributed version control systems or source code management (SCM) systems are available. There are a few open source SCM systems that have been widely used over the years:

- Concurrent Versions System (CVS)
- Apache Subversion
- Git
- Others that are used in particular niches (e.g., Bazaar, Mercurial) and a few commercial tools (Perforce, ClearCase)

Although each has its advantages and disadvantages, the use of Git in the Linux system build process (and projects based on Linux, such as the Android mobile environment), as well as the availability of sites like *github.com*, *gitlab.com*, and others, give Git a massive momentum over the others. I don't have statistics, but I suspect the number of projects in Git repositories probably exceeds the others combined. Several well-known organizations using Git are listed on the Git home page.[2]

For this reason, I have migrated all my public projects to GitHub; see my GitHub repository. To download the projects and get updates applied automatically, use Git to download them. Options include the following:

- Use the command-line Git client. If you are on any modern Unix or Linux system (including macOS), Git is either included or available in your ports or packaging or developer tools. On modern MS Windows systems, use `winget install git.git`. Finally, Git can also be downloaded for MS Windows, macOS, and Linux from the Git home page under Downloads.
- All modern IDEs have Git support built in (though IntelliJ doesn't include Git itself; it relies on the command-line Git client, possibly because the main Java implementation `jgit` is owned by Eclipse).
- Numerous standalone GUI clients.
- Even continuous integration servers such as Jenkins (see Recipe 2.15) have plug-ins available for updating a project with Git (and other popular SCM systems) before building them.

You will want to have one or more of these Git clients at your disposal to download my code examples. You could instead download the code examples as ZIP archive files, but then you wouldn't get updates! You can also view or download individual files from the GitHub page via a web browser.

---

# 1.4 Compiling, Running, and Testing with an IDE

## Problem

It is cumbersome to use several tools for the various development tasks.

## Solution

Use an Integrated Development Environment (IDE), which combines editing, testing, compiling, running, debugging, and package management.

## Discussion

Many programmers find that using a handful of separate tools—a text editor, a compiler, and a runner program, not to mention a debugger—is too many. An IDE *integrates* all of these into a single toolset with a graphical user interface. Many IDEs are available, and the better ones are fully integrated tools with their own copies of the compilers and virtual machines. Class browsers and other features of IDEs round out the ease-of-use feature sets of these tools. Today most developers use an IDE because of the productivity gains. Although I started as a command-line junkie, I do find that IDE features like the following make me more productive:

*Code completion*

> *Ian's Rule* here is that I never type more than three characters of any name that is known to the IDE; let the computer do the typing!

*Incremental compiling features*

Note and report compilation errors as you type, instead of waiting until you are finished typing.

*Refactoring*

The ability to make far-reaching yet behavior-preserving changes to a codebase without having to manually edit dozens of individual files.

Beyond that, I don't plan to debate the merits of IDE versus the command-line process; I use both modes at different times and on different projects. I'm just going to show a few examples of using a couple of the Java-based IDEs.

The most popular Java IDEs, which run on all mainstream computing platforms and quite a few niche ones, are *Eclipse*, *IntelliJ IDEA*, *VSCode*, and *NetBeans*. Eclipse was for a long time the most widely used by Java-only developers. Eclipse was created by IBM, which set up the Eclipse Software Foundation to host it; the ESF now hosts dozens of open source projects including JakartaEE, the follow-on to Java Enterprise Edition. The Eclipse Platform is also used as the basis of other tools such as SpringSource Tool Suite (STS) and IBM's Rational Application Developer (RAD). If you develop for Android, the Android tooling was originally based on Eclipse, but transitioned a decade or so ago to IntelliJ as the basis for Android Studio, which is now the standard IDE for Android and for Google's other mobile platform, Flutter.

IntelliJ IDEA is a project of JetBrains, and has both a free community edition and a pro ("Ultimate") edition. Note that JetBrains also provides the JetBrains Toolbox App for macOS, Windows, and Linux. This will install IntelliJ IDEA, Android Studio, PyCharm, and a host of other JetBrains IDEs, and allow you to update all the IDEs that are out-of-date with a single click.

Microsoft's VSCode is a later entrant, originally for .NET developers. VSCode is open source and, like the others, supports a variety of languages in addition to Java.

NetBeans was a startup company and was bought by Sun, which was then bought by Oracle, which in turn donated NetBeans to the Apache Software Foundation.

All these Java IDEs are plug-in based and offer a wide selection of optional and third-party plug-ins to enhance the IDE, such as supporting other programming languages, frameworks, and file types. For Eclipse, use the Eclipse Marketplace, near the bottom of the Help menu. For IntelliJ IDEA, use the Settings → Plugins menu item.

While the following paragraph shows creating and running a program with Eclipse, the IntelliJ IDEA and NetBeans IDEs all offer similar capabilities. All three of these IDEs are written mostly in Java. IntelliJ uses some OS-specific native code, and is also the basis of half a dozen other IntelliJ IDEs including PyCharm, CLion, and others. Eclipse has its own GUI toolkit, unfortunately not written in Java. As a result, on geekly systems like OpenBSD we have IntelliJ IDEA and NetBeans but not Eclipse.

All IDEs do basically the same thing for you when getting started. The example in Figure 1-1 shows starting a new project.

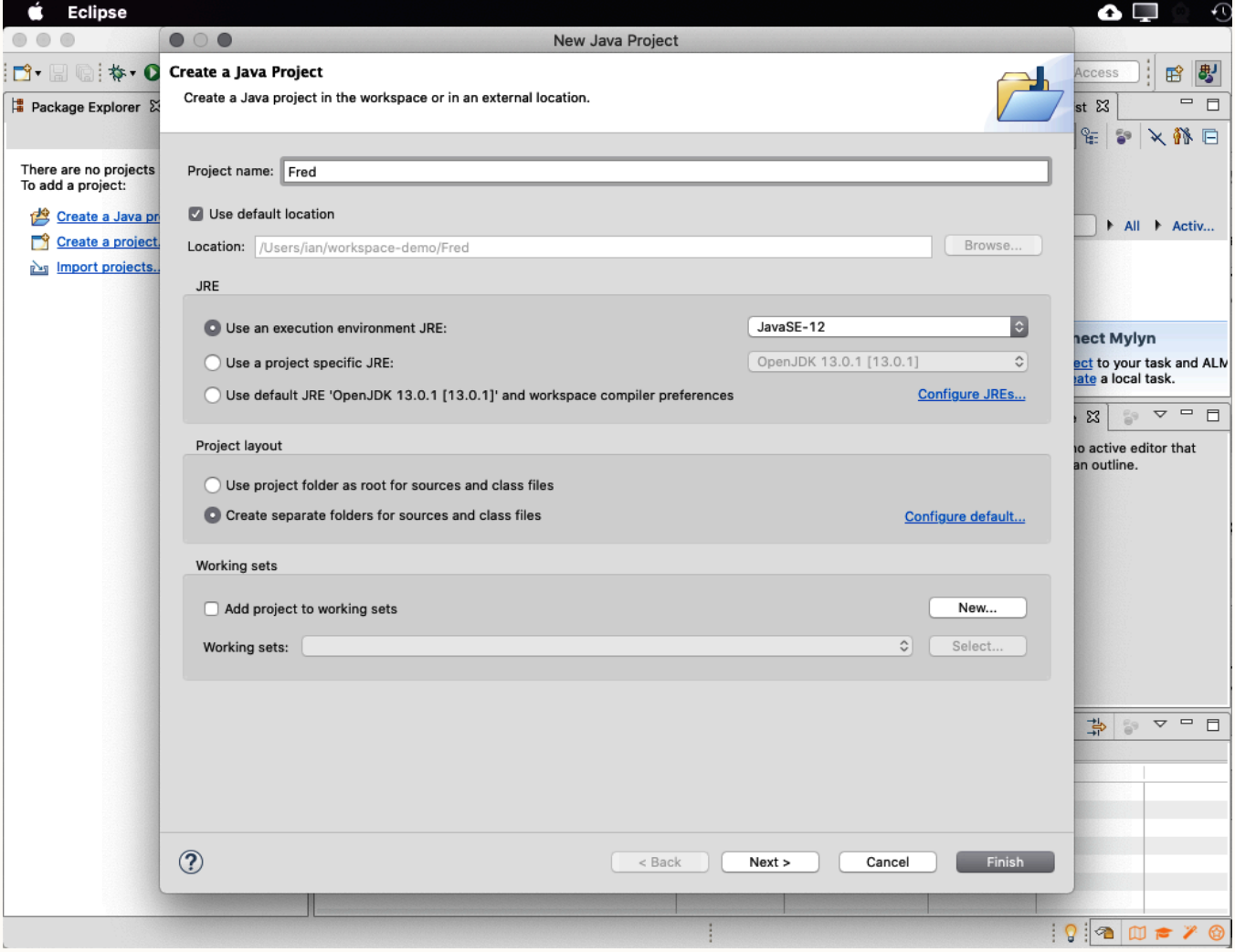



Figure 1-1. Starting a new project with the Eclipse New Java Project Wizard

The Eclipse New Java Class Wizard shown in Figure 1-2 shows creating a new class.

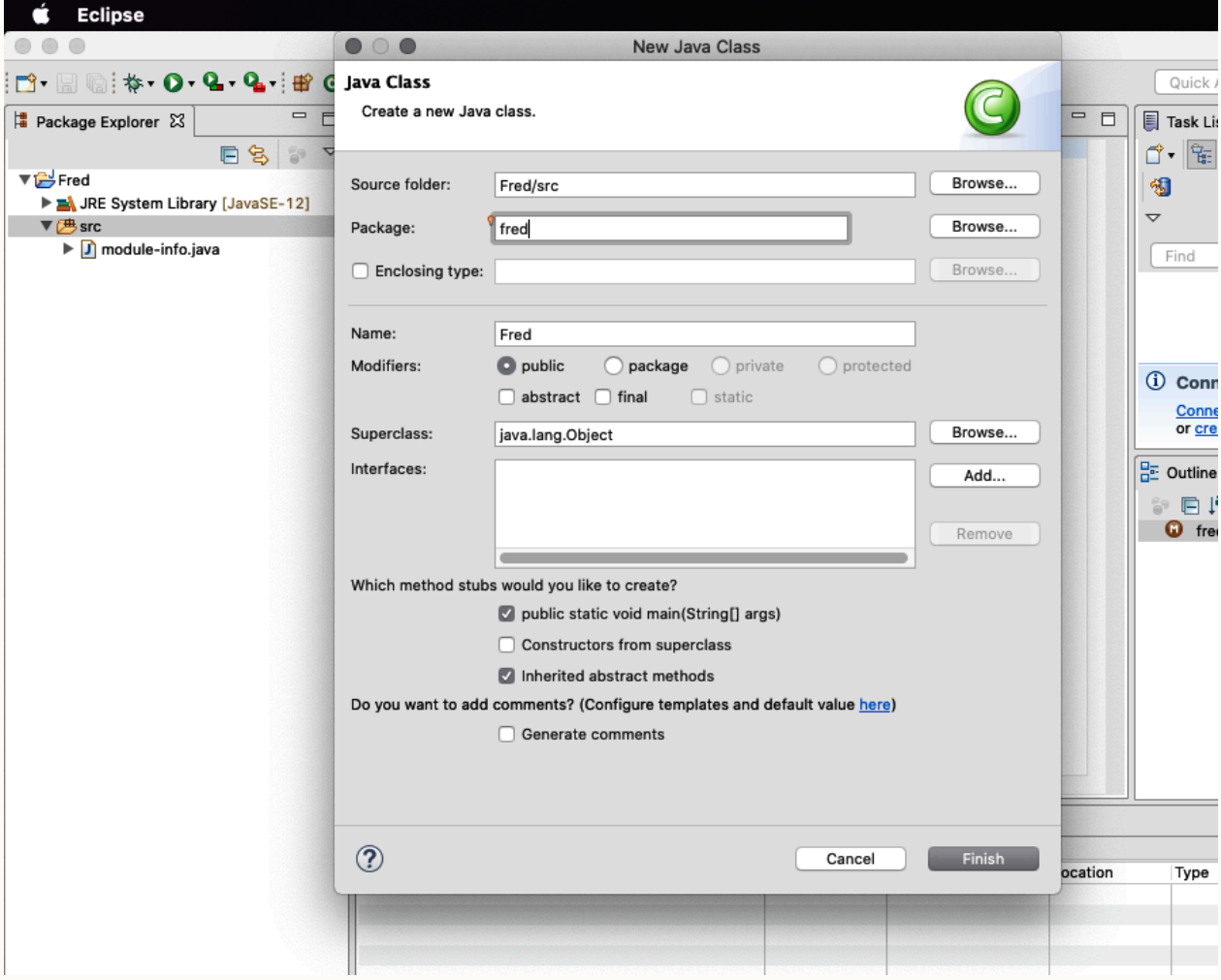



Figure 1-2. Creating a new class with the Eclipse New Java Class Wizard

Eclipse, like all modern IDEs, features a number of refactoring capabilities, shown in Figure 1-3.

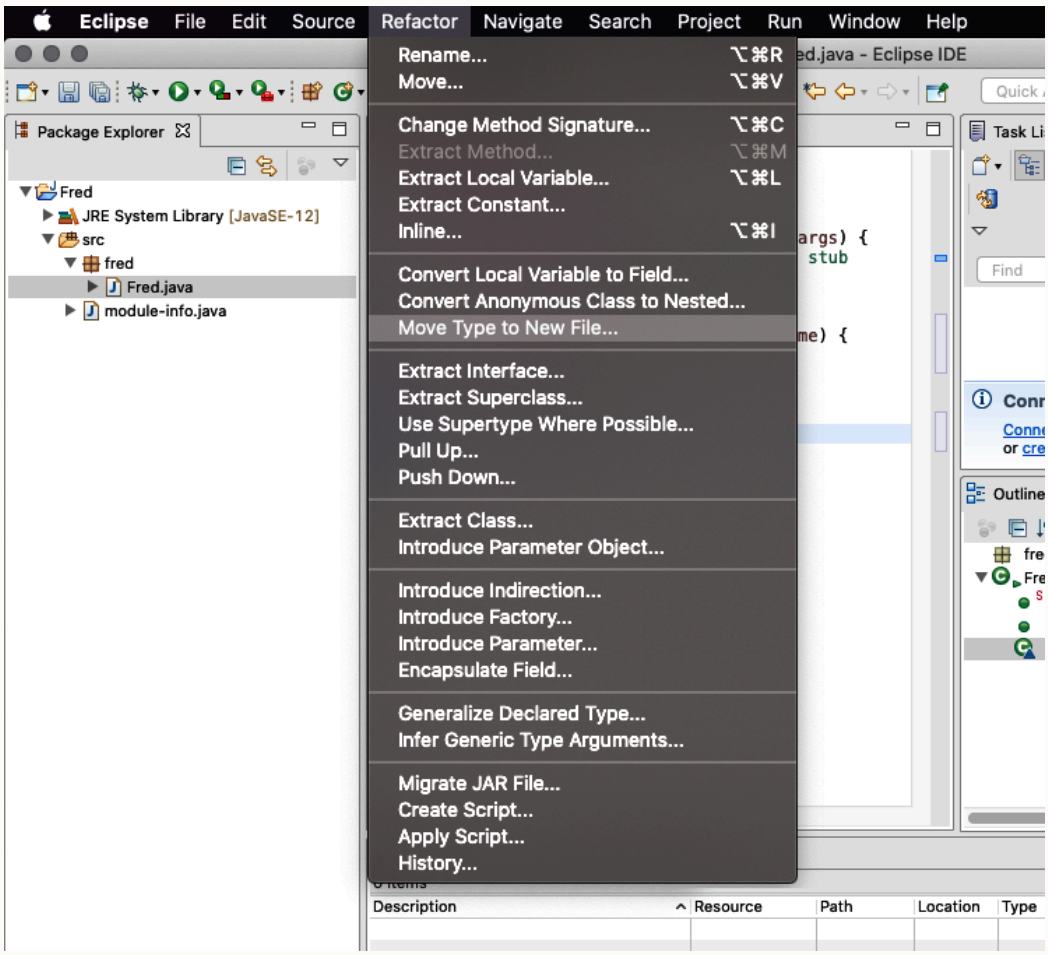



Figure 1-3. Refactoring in Eclipse

And, of course, all the IDEs allow you to run and/or debug your application. Figure 1-4 shows running a Hello, World application using IntelliJ IDEA, and Figure 1-5 shows a similar application in NetBeans.

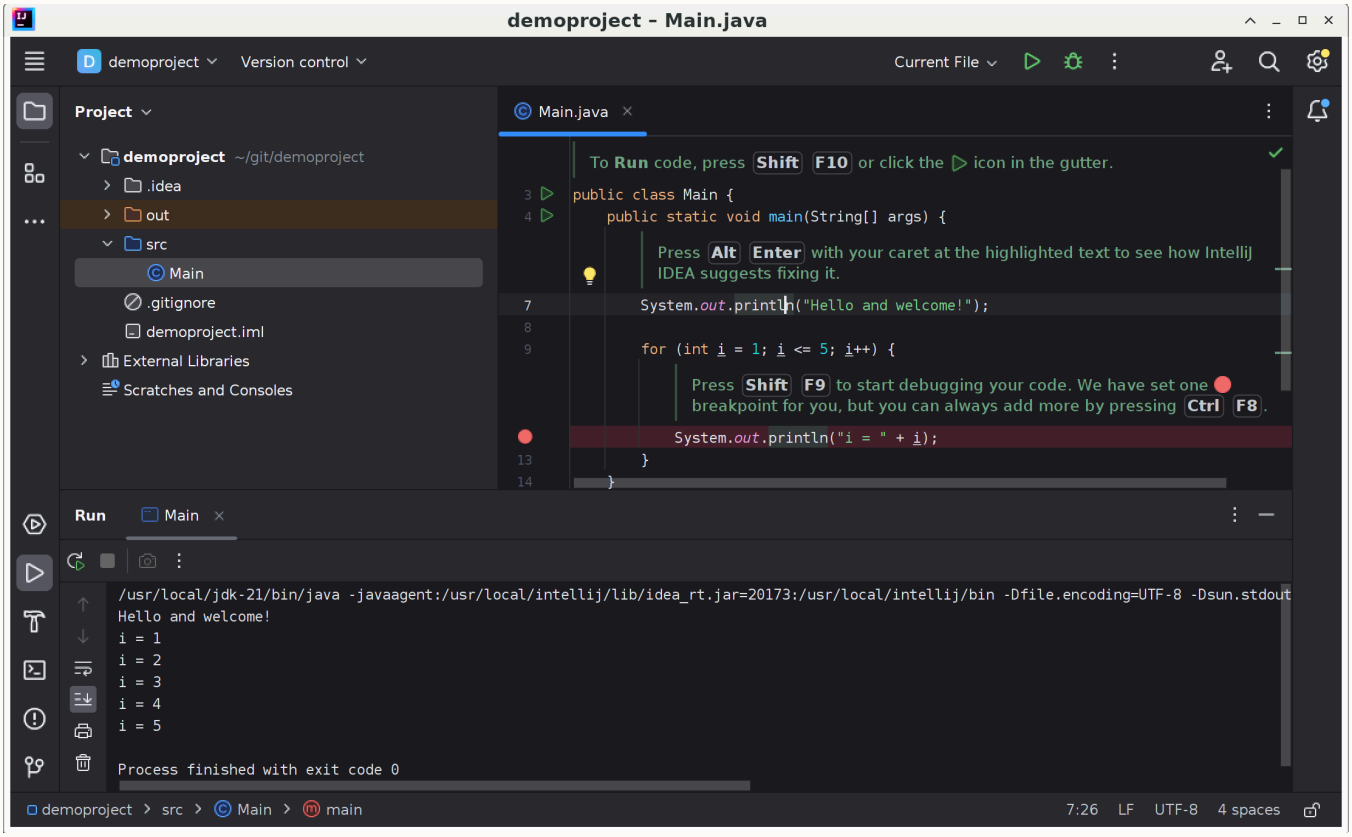



Figure 1-4. IntelliJ run program output

macOS (Release 10.x of the OS) is built upon a BSD Unix (and "Mach") base. As such, it has a regular command line (the Terminal application, hidden away under */Applications/Utilities*) and most of the traditional Unix command-line tools for developers and other users. Mac fans can use one of the many full IDE tools discussed in Recipe 1.4. The main IDE from Apple, Xcode, doesn't really work for Java developers.

Microsoft's open source VSCode has been getting some attention in Java circles lately, but it's not a Java-specific IDE. Give it a try if you like.

How do you choose an IDE? Perhaps it will be dictated by your organization or chosen by majority vote of your fellow developers. Given that all three major IDEs (Eclipse, NetBeans, and IntelliJ) can be downloaded for free, and two of them are open source, why not try them all and see which one best fits the kind of development you do? Regardless of what platform you use to develop Java, there are enough Java IDEs from which to choose. Once you have chosen one, do spend time learning the keyboard shortcuts; they will save you days of time a year if used effectively. Eclipse users should check out my article "Eclipse Shortcuts You Cannot Afford to Ignore".

Note that IntelliJ IDEA puts many "helper" comments on screen that don't appear in the source code being edited.

NetBeans has most of the same capabilities. Figure 1-5 shows a program being run in NetBeans.

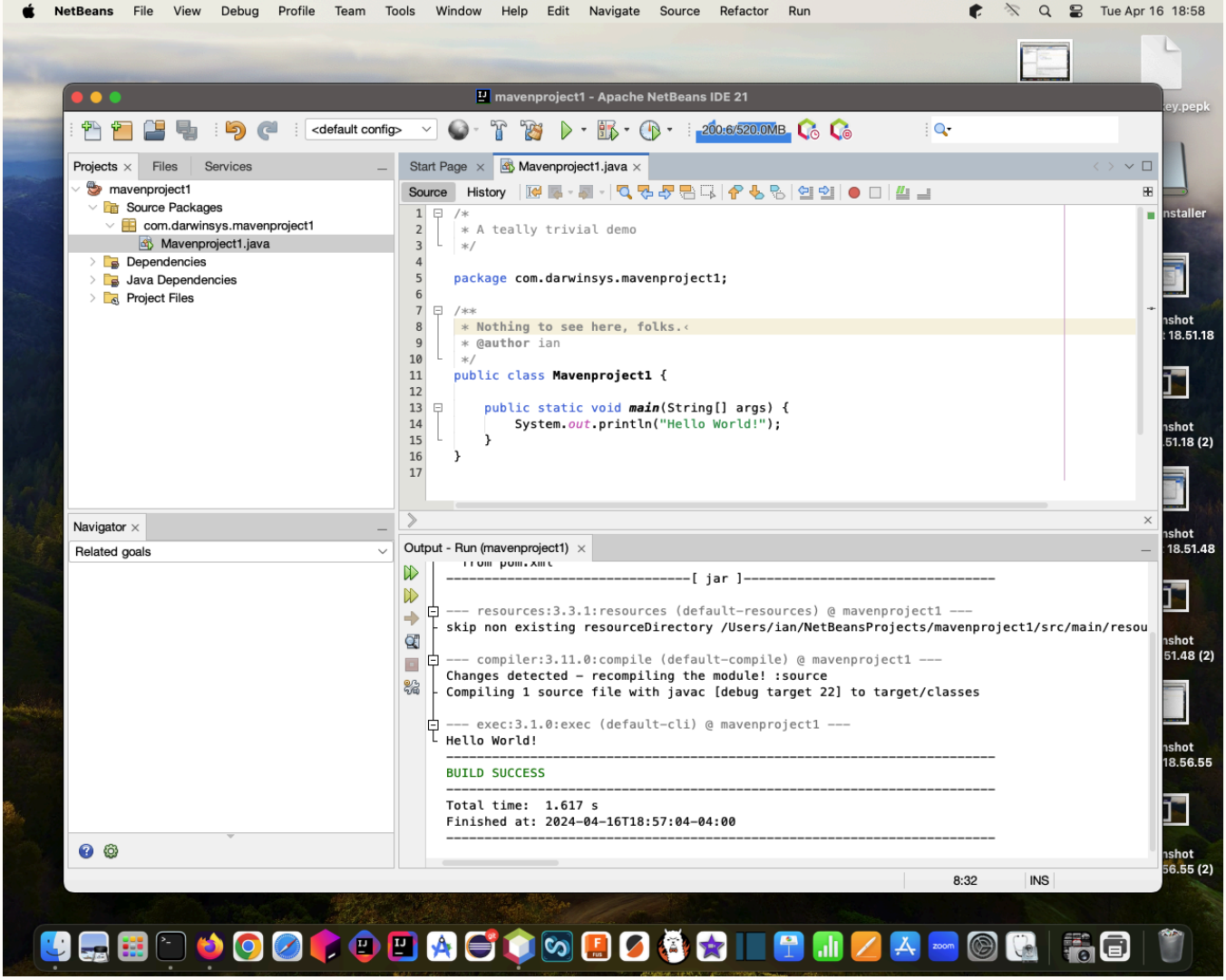



Figure 1-5. NetBeans run program output

## See Also

See Table 1-4 for the websites for each major IDE.

Table 1-4. The three major Java IDEs and their websites

| Product name | Project URL | Note |
|---|---|---|
| Eclipse | *https://eclipse.org* | Basis of STS, RAD |
| IntelliJ IDEA | *https://jetbrains.com/idea* | Basis of Android Studio |
| NetBeans | *https://netbeans.apache.org* | Runs anywhere Java SE does |

For Eclipse, I have some useful information at my website (*https://darwinsys.com/java*). That site includes a list of shortcuts to aid developer productivity.

# 1.5 Exploring Java with JShell 11

## Problem

You want to try out Java expressions and APIs quickly, without having to create and run a source file every time.

## Solution

Use JShell, Java's interactive REPL (read-evaluate-print loop) interpreter.

## Discussion

Starting with Java 11, JShell is included as a standard part of the JDK. It allows you to enter Java statements and have them evaluated without the bother of creating a class and a main program. You can use it for quick calculations, to try out an API to see how it works, or for almost any purpose; if you find an expression you like, you can copy it into a regular Java source file and make it permanent. JShell can also be used as a scripting language over Java, but the overhead of starting the JVM means that it won't be as fast as AWK, Perl, or Python for quick scripting.

REPL programs are very convenient, and they are hardly a new idea (LISP languages from the 1950s included them). You can think of command-line interpreters (CLIs) such as the bash or ksh shells on Unix/Linux, or `Command.com` and PowerShell on Microsoft Windows, as REPLs for the system as a whole. Many interpreted languages like Ruby and Python can also be used as REPLs. Java has its own REPL, *JShell*. Here's an example of its use:

```
$ jshell
|  Welcome to JShell -- Version 11.0.2
|  For an introduction type: /help intro

jshell> "Hello"
$1 ==> "Hello"

jshell> System.out.println("Hello");
Hello

jshell> System.out.println($1)
Hello

jshell> "Hello" + sqrt(57)
|  Error:
|  cannot find symbol
|    symbol:   method sqrt(int)
|  "Hello" + sqrt(57)
|            --
|

jshell> "Hello" + Math.sqrt(57)
$2 ==> "Hello7.54983443527075"

jshell> String.format("Hello %6.3f", Math.sqrt(57)
   ...> )
$3 ==> "Hello  7.550"

jshell> String x = Math.sqrt(22/7) + " " + Math.PI +
   ...> " and the end."
x ==> "1.7320508075688772 3.141592653589793 and the end."

jshell>
```

You can see some obvious features and benefits here:

- The value of an expression is printed without needing to call `System.out.println` every time, but you can call it if you like.
- Values that are not assigned to a variable get assigned synthetic identifiers, like `$1`, that can be used in subsequent statements.
- The semicolon at the end of a statement is optional (unless you type more than one statement on a line).
- If you omit a close quote, parenthesis, or other punctuation, JShell will just wait for you, giving a continuation prompt ( `...` ).
- If you make a mistake, you get a helpful message immediately.
- If you do make a mistake, you can use "shell history" (i.e., up arrow) to recall the statement so you can repair it.
- Not shown but worth knowing: you can get API completion with a single tab, as in shell filename completion.
- You can get the relevant portion of the Javadoc documentation on known classes or methods with just a double tab.

JShell is also useful in prototyping Java code. For example, I wanted one of those health-themed timers that reminds you to get up and move around a bit every half hour:

```
$ jshell
|  Welcome to JShell -- Version 11.0.2
|  For an introduction type: /help intro

jshell> while (true) { sleep (30*60); JOptionPane.showMessageDialog(null,
  "Move it"); }
|  Error:
|  cannot find symbol
|    symbol:   method sleep(int)
|  while (true) { sleep (30*60); JOptionPane.showMessageDialog(null, "Move it");}
|              ^---^
|  Error:
|  cannot find symbol
|    symbol:   variable JOptionPane
|  while (true) { sleep (30*60); JOptionPane.showMessageDialog(null, "Move it");}
|                          ^---------^

jshell> import javax.swing.*;

[At this point I simply hit up-arrow and return]

jshell> while (true) { Thread.sleep (30*60 * 1000);
  JOptionPane.showMessageDialog(null, "Move it"); }

jshell> ^D
```

I then put the final working version into a Java file called *BreakTimer.java*, put a `class` statement and a `main()` method around the main line, told the IDE to reformat the whole thing, and saved it into my *darwinsys-api* repository.

So go ahead and experiment with JShell. Read the built-in introductory tutorial for more details! When you get something you like, either use `/save`, or copy and paste it into a Java program and save it.

Read more about JShell at the OpenJDK JShell Tutorial.

# 1.6 Using CLASSPATH Effectively

## Problem

You need to keep your class files in a common directory or library, or you're wrestling with `CLASSPATH`.

## Solution

Set `CLASSPATH` to the list of directories and/or JAR (Java ARchive) files that contain the classes you want.

## Discussion

`CLASSPATH` is a list of class directories containing compiled class files in any of a number of directories or JAR files. Just like the `PATH` your system uses for finding programs, the `CLASSPATH` is used by the Java runtime to find classes. Even when you type something as simple as `java HelloWorld`, the Java interpreter looks in each place named in your `CLASSPATH` until it finds a match. Let's work through an example.

The `CLASSPATH` can be set as an environment variable the same way you set other environment variables, such as your `PATH` environment variable. However, it's usually preferable to specify the `CLASSPATH` for a given command on the command line:

```
C:\> java -classpath c:\ian\classes starting.HelloWorld
```

We can use the `-d` option of `javac` to compile classes into such a directory:

```
$ javac -d $HOME/classes HelloWorld.java
$ java -cp $HOME/classes starting.HelloWorld
Hello, world!
```

Suppose your `CLASSPATH` were set to *C:\classes;.* on Windows or *~/classes:.* on Unix or Mac. Suppose you had just compiled a source file named *HelloWorld.java* (with no package statement) into *HelloWorld.class* in the default directory (which is your current directory) and tried to run it. On Unix, if you run one of the kernel tracing tools (`trace`, `strace`, `truss`, or `ktrace`), you would probably see the Java program `open` or `stat` or `access` the following files:

- Some file(s) in the JDK directory
- Then *~/classes/HelloWorld.class*, which it probably wouldn't find
- Finally, *./HelloWorld.class*, which it would find, open, and read into memory

The vague "some file(s) in the JDK directory" is release dependent. You should not mess with the JDK files, but if you're curious, you can find them in the system properties (see "Getting information from system properties").

The reason I and others suggest *not* setting `CLASSPATH` as an environment variable is that we don't like surprises. It's easy to add a JAR to your `CLASSPATH` and then forget that you've done so; a program might then work for you but not for your colleagues, due to their being unaware of your hidden dependency. And if you add a new version to `CLASSPATH` without removing the old version, you may run into conflicts.

Note also that providing the `-classpath` argument causes the `CLASSPATH` environment variable to be ignored.

If you still want to set `CLASSPATH` as an environment variable, you can. Suppose you had also installed the JAR file containing the supporting classes for programs from this book, *darwinsys-api.jar* (the actual filename if you download it may have a version number as part of the filename). You might then set your `CLASSPATH` to *C:\classes;C:\classes\darwinsys-api.jar;.* on Windows or *~/classes:~/classes/darwinsys-api.jar:.* on Unix.

Notice that you *do* need to list the full name of the JAR file explicitly. Unlike a single class file, placing a JAR file into a directory listed in your `CLASSPATH` does not make it available.

While these examples show explicit use of `java` with `-classpath`, it is generally more convenient (and reproducible) to use a build tool such as Maven (Recipe 2.4) or Gradle (Recipe 2.5), which automatically provides the `CLASSPATH` for both compilation and execution.

Note that Java 9 and later also have a module path (environment variable `MODULEPATH`, command-line argument `--module-path entry[:,...]`) with the same syntax as the classpath. The module path contains code that has been modularized; the Java Platform Module System is discussed in Recipes 2.2 and 2.3.

# 1.7 Documenting Classes with Javadoc

## Problem

You have heard about this thing called *code reuse* and would like to promote it by informing developers (including yourself, later) how to use your classes.

## Solution

Use Javadoc. Write the comments when you write the code.

## Discussion

Javadoc is one of the great inventions of the early Java years. Like so many good things, it was not wholly invented by the Java folks; earlier projects such as Knuth's Literate Programming had combined source code and documentation in a single source file. But the Java folks did a good job on it, and it came along at the right time. Javadoc is to Java classes what *man pages* are to Unix, or what Windows Help is to Windows applications: it is a standard format that everybody expects to find and knows how to use. Learn

it (this takes some discipline: learn how others use it effectively). Use it. Write it. Maintain it. Live long and prosper (well, perhaps that's not guaranteed). All that HTML documentation that you learned from writing Java code, the complete reference for the JDK—did you think they hired dozens of tech writers to produce it? Nay, that's not the Java way. Java's developers wrote the documentation comments as they went along, and when the release was made, they ran `javadoc` on all the zillions of public classes and generated the documentation bundle at the same time as the JDK. You can, should, and really must do the same when you are preparing classes for other developers to use.

All you have to do to use Javadoc is to put special *Javadoc comments* into your Java source files. These are similar to multiline Java comments, but they begin with a slash and *two* stars and end with the normal star-slash. Javadoc comments must appear immediately before the definition of the class, method, or field that they document; if placed elsewhere, they are ignored.

A series of keywords, prefixed by the at sign, can appear inside Javadoc comments in certain contexts. Some are contained in braces. The keywords are listed in Table 1-5.

Table 1-5. Javadoc keywords

| Keyword | Use |
|---|---|
| `@author` | Author name(s) |
| {@code *text*} | Displays text in code font without HTML interpretation |
| `@deprecated` | Explains deprecation warning (when, why, what to replace with) |
| {@docroot} | Refers to the root of the generated documentation tree |
| `@exception` | Alias for `@throws` |
| {@inheritDoc} | Inherits documentation from nearest superclass/superinterface |
| `@link` | Generates inline link to another class or member |
| {@link *ref* label} | Generates inline link with label to another class or member |
| `@linkplain` | Same as `@link` but displays in plain text |
| {@literal *text*} | Displays text without interpretation |
| `@param` *name description* | Argument name and meaning (methods only) |
| `@return` | Returns value |
| `@see` | Generates cross-reference link to another class or member |
| `@serial` | Describes serializable field |
| `@serialData` | Describes order and types of data in serialized form |
| `@serialField` | Describes serializable field |
| `@since` | Tells the JDK version in which introduced (primarily for Sun use) |
| `@snippet` **18** | Includes code snippet |
| `@throws` | Exception class and conditions under which thrown |
| {@value [*ref*]} | Displays values of this or another constant field |
| `@version` | Version identifier |

Example 1-2 is a somewhat contrived example that shows some common Javadoc keywords in use. The output of running this through `javadoc` is shown in a browser in Figure 1-6.

**Example 1-2. main/src/main/java/javadoc/JavadocDemo.java**

```
/**
 * Construct the GUI
 * @throws java.lang.IllegalArgumentException if constructed on a Sunday.
 */
```

```
  public JavadocDemo() {
    // We create and add a pushbutton here,
    // but it doesn't do anything yet.
    Button b = new Button("Hello");
    add(b);          // connect Button into component
    // Totally capricious example of what you should not do
    if (LocalDate.now().getDayOfWeek() == DayOfWeek.SUNDAY) {
      throw new IllegalArgumentException("Never On A Sunday");
    }
  }

  /** paint() is an AWT Component method, called when the
   *  component needs to be painted. This one just draws colored
   * boxes in the window.
   *
   * @param g A java.awt.Graphics that we use for all our
   * drawing methods.
   */
  @Override
  public void paint(Graphics g) {
    // ...
```

The `javadoc` tool works fine for one class but really comes into its own when dealing with a package or collection of packages. You can provide a package summary file for each package, which will be incorporated into the generated files. Javadoc generates thoroughly interlinked and crosslinked documentation, just like that which accompanies the standard JDK. There are numerous command-line options; I normally use `-author` and `-version` to get it to include these items, and often `-link` to tell it where to find the standard JDK to link to. There is also the output directory argument `-d` to avoid cluttering the source folder with the dozens of files that Javadoc creates.

Run `javadoc -help` for a complete list of options, or see the full documentation online at Oracle's website. Figure 1-6 shows one view of the documentation that the class shown in Example 1-2 generates when run as the following:

```
$ javadoc -author -version JavadocDemo.java
```

If you run this with Java 9+, it will also include a fully functional search box, as shown in the upper right of Figure 1-6. This is implemented in JavaScript, so it should work in any modern browser.

Be aware that quite a few files are generated, and one of the generated files will have the same name as each class, with the extension *.html*. Since you probably wish to avoid cluttering up your source directories with the generated files, the `-d` *directorypath* option for Javadoc allows you to place the generated files into the specified directory.

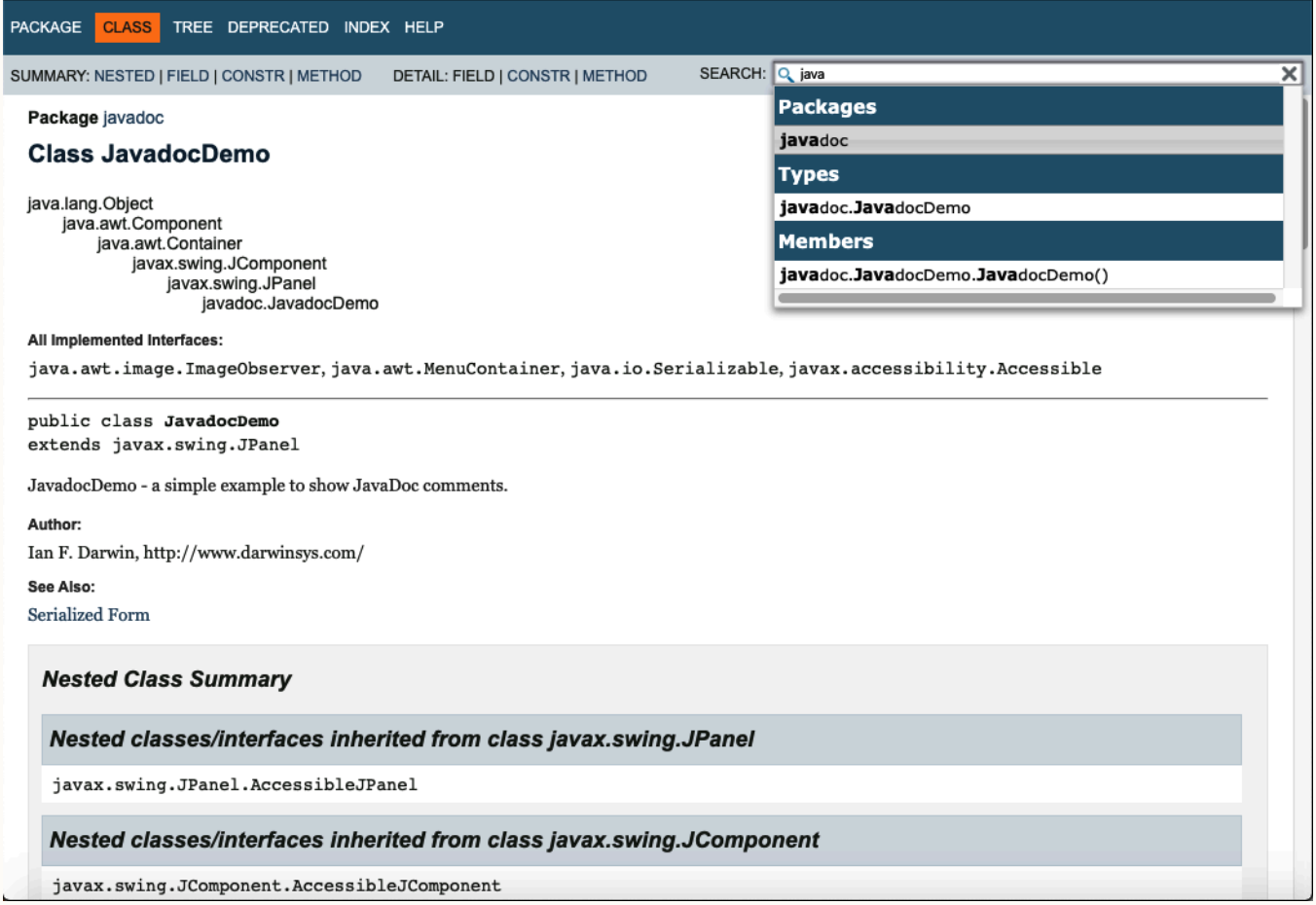



Figure 1-6. Javadoc opened in a browser

## Snippets 18

As of Java 18, Javadoc can include internal or external "code snippets" that provide a brief code sample showing use of the class or method being documented. Example 1-3 shows an "internal" snippet, where the sample code is embedded in the `@snippet` tag (the tag and its content is required to be surrounded by a `{...}` pair).

**Example 1-3. main/src/main/java/javadoc/JavadocDemo.java**

```
/**
 * A simple demo method. Typical usage:
 * {@snippet lang="java" :
 *  var demo = new JavadocDemo();
 *  demo.demo(42); // or some other int
 * }
 * @param i The value to be processed.
 */
public void demo(int i) {
  System.out.printf("Demo value is %d\n", i);
}
```

When processed via `javadoc`, the result in Figure 1-7 is created. Note the convenient Copy button on the right!

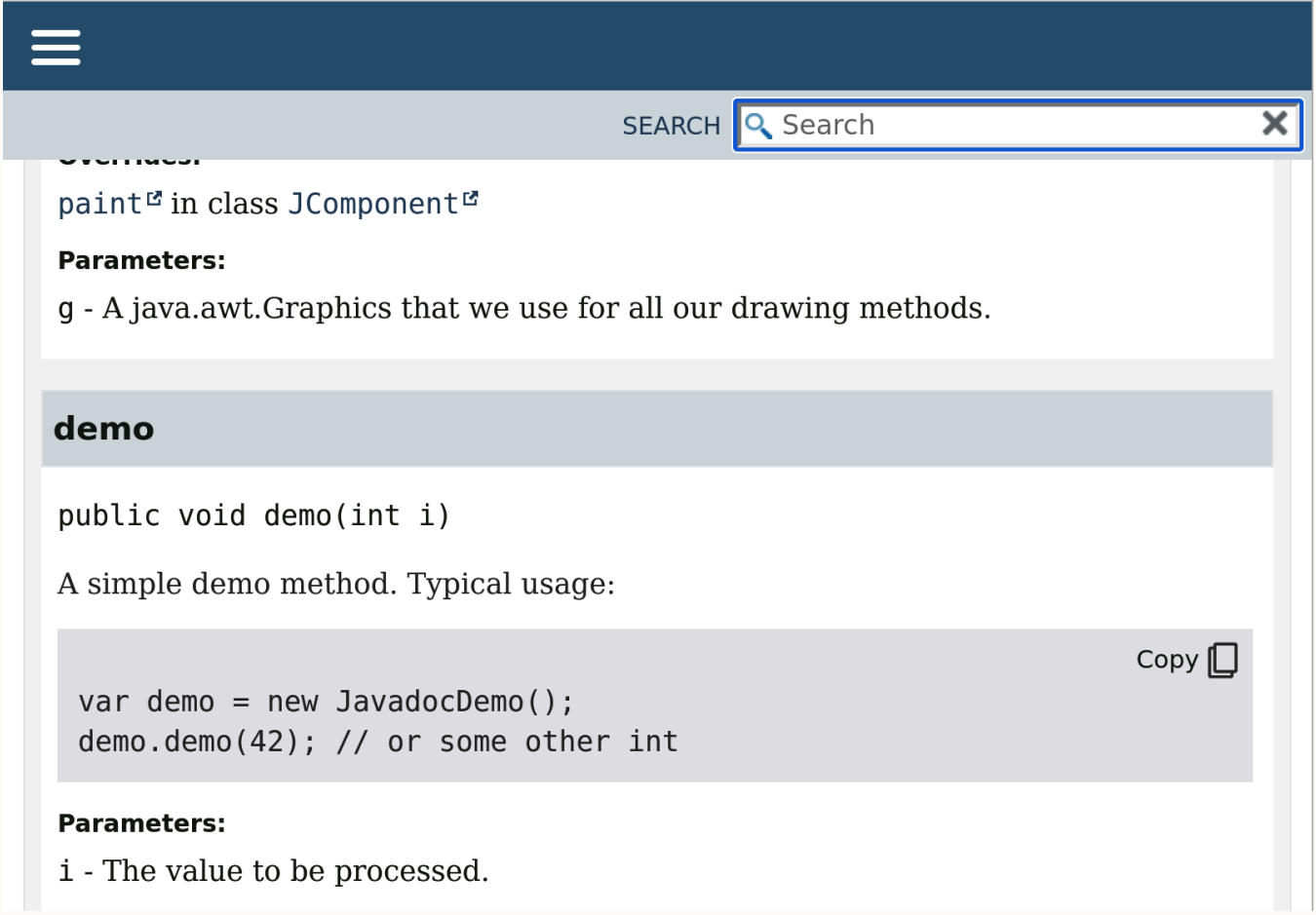



Figure 1-7. An internal snippet

The content of snippets is not limited to Java code; any text format can be used. Internal snippets cannot contain /…/ comments, nor an excess of mismatched "}" characters.

External snippets are files that are included in the generated documentation. They can be in a `snippet-files` subdirectory of the directory containing the source code or in an external folder (which must be passed as `--snippet-path` to the `javadoc` command). They can optionally contain subset "regions," delimited by `// @start region=name` and ending with `// @end [region=name]`. If the code is in another language than Java, its comment convention can be used. If the region argument isn't provided on the `@end`, the most recent region is ended. Example 1-4 shows an external snippet.

**Example 1-4. main/src/main/java/javadoc/JavadocDemo.java**

```
/**
 * A simple method. See this note:
 * {@snippet lang="python" file="snyde_comment.py"}
 * @param i The value to be processed.
 */
public void demo2(int i) {
  System.out.printf("Demo value is %d\n", i);
}
```

---

NOTE

External snippets in the default location (in `snippet-files` under the package source directory) seem only to be found if you run Javadoc from the top-level directory, as is normally done with `mvn javadoc:javadoc`.

---

There's considerably more capability to the snippet mechanism. For more details please refer to the JEP 413 documentation.

## Markdown 23

Java 23 introduced an entirely new mechanism for writing Javadoc, through the use of Markdown, a simple document formatting language. I would have preferred AsciiDoc, but they didn't ask me, so JEP 467 specifies Markdown.

Markdown Javadoc is introduced by a triple slash ( `///` ) on each line, rather than the traditional `/** ... */` comment. You still use most of the keywords from Table 1-5. However, Markdown simplifies document formatting, so instead of using HTML tags, you can just write text, with empty lines for paragraph breaks, leading dashes for list items, and square brackets ( `[]` ) for JDK links or `[link text]` (*https://link_url_address*) for external links.

Example 1-5 shows the same code as Example 1-2 but with most of the Javadoc changed to Markdown. In the online copy of this file, one snippet example has been left in the traditional format to show that you can mix and match both formats in the same file.

**Example 1-5. main/src/main/java/javadoc/JavadocMdDemo.java**

```
///
/// JavadocMdDemo - a MarkDown example to show JavaDoc comments.
///
/// In Java 23, JEP 467 specifies [Markdown](https://en.wikipedia.org/wiki/Markdown)
/// as an alternate implementation language to prepare Javadoc.
///
/// Markdown Javadoc uses three slashes per line instead of the /** ... */ by
/// traditional JavaDoc. This format has some advantages:
/// - it's easier to write, and to read in the code
/// - bullet points like this are easier to format
/// - etc.
///
/// @author Ian Darwin https://darwinsys.com/java
///
public class JavadocMdDemo extends JPanel {

 private static final long serialVersionUID = 1L;

 ///
 /// Construct the GUI
 /// @throws java.lang.IllegalArgumentException if constructed on a Sunday.
 ///
 public JavadocMdDemo() {
     // ...
```

When run, it looks like Figure 1-8.

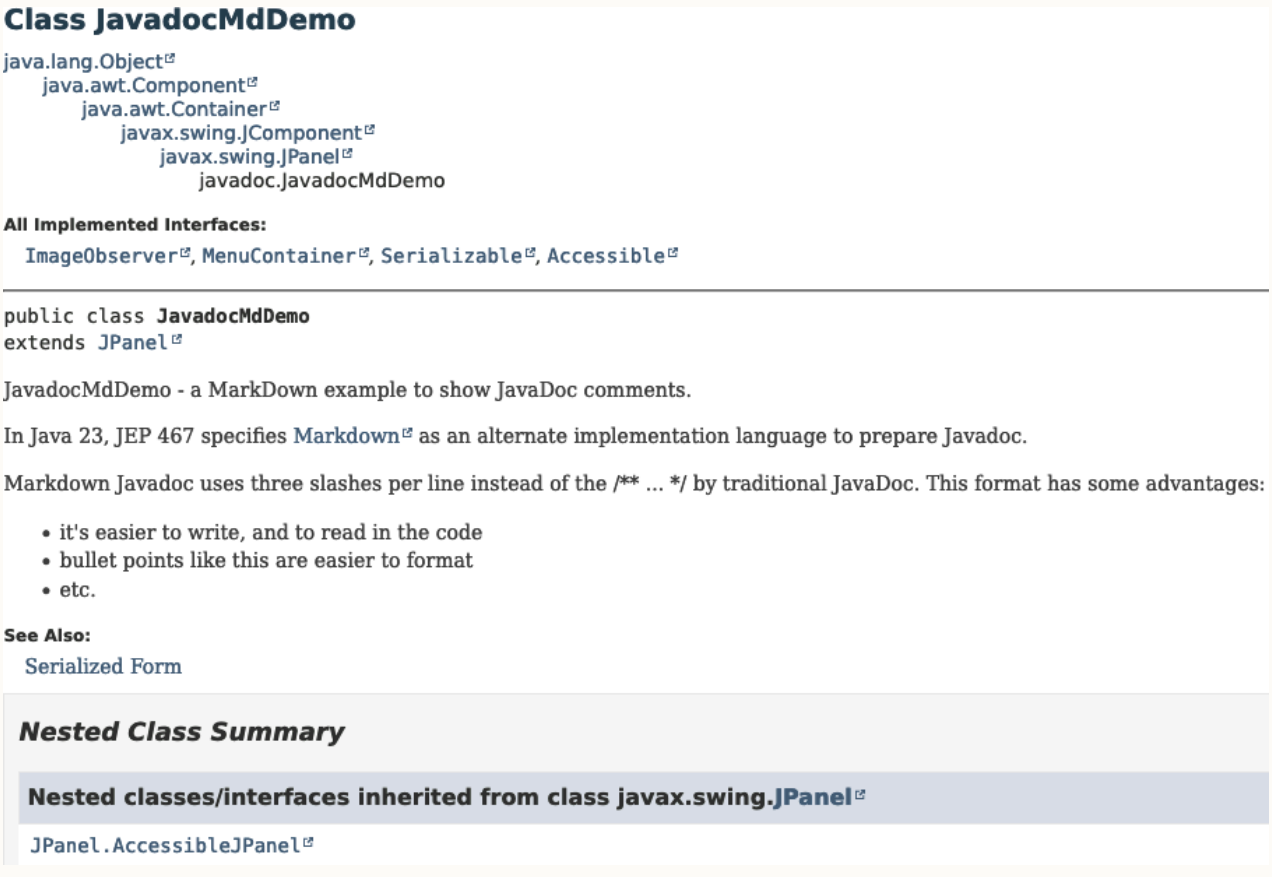



Figure 1-8. Javadoc prepared with Markdown

### See Also

The `javadoc` tool has numerous other command-line arguments. If documentation is for your own use only and will not be distributed, you can use the `-link` option to tell it where your standard JDK documentation is installed so that links can be generated to standard Java classes (like `String`, `Object`, and so on). If documentation is to be distributed, you can omit `-link` or use `-link` with a URL to the appropriate Java API page on Oracle's website. See the online tools documentation for all the command-line options.

The output that Javadoc generates is fine for most purposes. It is possible to write your own `Doclet` class to make the Javadoc program into a class documentation verifier, a Java-to-other-format (such as Java-to-AsciiDoc) documentation generator, or whatever you like. See the Javadoc tools documentation that comes with the JDK for documents and examples, or go to Oracle's website.

### Javadoc and Dash/Zeal

Dash is a Mac-centric documentation viewer. There is an open source implementation called Zeal, as well as plug-ins for several popular IDEs and editors including the popular Java IDE IntelliJ IDEA. Dash's sponsor Kapeli provides over 200 sets of documents for free download. You can quite easily package a set of Javadoc pages into a docset that will be usable by both these programs; use Kapeli's tool for macOS or this version for other platforms. This is great for internal use. If your software is being made available to the public or to a decent-sized customer base, you may want to publish the docset back to Dash.

### Javadoc Versus JavaHelp

Javadoc is for programmers using your classes; for a GUI application, users of desktop applications will appreciate standard online help. This is the role of the JavaHelp API, which is not covered in this book but is fully explained in *Creating Effective JavaHelp* by Kevin Lewis (O'Reilly). JavaHelp is another useful specification that was left to coast during the Sun sellout to Oracle; what remains of it—the old source code—is hosted at javahelp.

## 1.8 Beyond Javadoc: Annotations/Metadata

### Problem

You want to generate not just documentation from your source code, but also other code artifacts. You want to mark code for additional compiler verification.

### Solution

Use the Java annotations, or metadata, facility.

### Discussion

The continuing success of the open source tool XDoclet—originally used to generate the tedious auxiliary classes and deployment descriptor files for the widely criticized EJB2 framework—led to a demand for a similar mechanism in standard Java. Java *annotations* were the result. The annotation mechanism uses an interface-like syntax, in which both declaration and use of annotations use the name preceded by an at character (`@`). This was chosen, according to the designers, to be reminiscent of "Javadoc tags, a preexisting ad hoc annotation facility in the Java programming language." Javadoc is ad hoc only in the sense that its `@` tags were never fully integrated into the language; most were ignored by the compiler, but `@deprecated` was always understood by the compiler.

Annotations can be read at runtime by use of the Reflection API; this is discussed in Recipe 17.12, where I also show you how to define your own annotations. Annotations can also be read post-compile time by tools such as code generators (and others to be invented, perhaps by you, gentle reader!).

Annotations are also read by `javac` at compile time to provide extra information to the compiler.

For example, a common coding error is overloading a method when you mean to override it, by mistakenly using the wrong argument type. Consider overriding the `equals` method in `Object`. If you mistakenly write:

```
public boolean equals(MyClass obj) {
   ...
}
```

then you have created a new overload that will likely never be called, and the default version in `Object` will be called. To prevent this, an annotation included in `java.lang` is the `Override` annotation. This has no parameters but is simply placed before the method call, like this:

```
/**
 * AnnotationOverrideDemo - Simple demonstation of Metadata being used to
 * verify that a method does in fact override (not overload) a method
 * from the parent class. This class provides the method.
 */
abstract class Top {
   public abstract void myMethod(Object o);
}

/** Simple demonstration of Metadata being used to verify
 * that a method does in fact override (not overload) a method
 * from the parent class. This class is supposed to do the overriding,
 * but deliberately introduces an error to show how the modern compiler
 * behaves
 */
class Bottom {

   @Override
   public void myMethod(String s) {    // EXPECT COMPILE ERROR
      // Do something here...
   }
}
```

Attempting to compile this results in a compiler error that the method in question does not override a method, even though the annotation says it does; this is a fatal compile-time error:

```
C:> javac AnnotationOverrideDemo.java
AnnotationOverrideDemo.java:16: method does not override a method
         from its superclass
      @Override public void myMethod(String s) {     // EXPECT COMPILE ERROR
       ^
1 error
C:>
```

As it should be.

# 1.9 Packaging and Running JAR Files

## Problem

You want to create a Java archive (JAR) file from your package (or any other collection of files). You want to be able to run your program from the JAR file.

## Solution

Use the command-line `jar` tool. Or use a build tool that will do the "jarring" for you. Create the JAR file with a `Main-Class:` entry in its manifest; run the program with the `java -jar` option.

## Discussion

The `jar` archiver is Java's standard tool for building archives. Archives serve the same purpose as the program libraries that some other programming languages use. Java normally loads its standard classes from archives, a fact you can verify by running a simple "Hello, World" program with the `-verbose` option:

```
java -verbose HelloWorld
```

Creating an archive is a simple process. The `jar` tool takes several command-line arguments: the most common are `c` for create, `t` for table of contents, and `x` for extract. The archive name is specified with `-f` and a filename. The options are followed by the files and directories to be archived, like this:

```
jar cvf /tmp/MyClasses.jar .
```

The dot at the end is important; it means the current directory. This command creates an archive of all files in the current directory and its subdirectories into the file */tmp/MyClasses.jar*.

Most applications of JAR files depend on an extra file that is always present in a true JAR file, called a *manifest*. This file always lists the contents of the JAR and their attributes; you can add extra information into it. The attributes are in the form `name: value`, as used in email headers, properties files (see Recipe 7.10), and elsewhere. Some attributes are required by the application, whereas others are optional. For example, running a main program directly from a JAR requires a `Main-Program` header. You can even invent your own attributes, such as the following:

```
MySillyAttribute: true
MySillynessLevel: high (6'2")
```

You store this in a file called, say, *manifest.stub*,[3] and pass it to `jar` with the `-m` switch. `jar` includes your attributes in the manifest file it creates:

```
jar -cv -m manifest.stub -f /tmp/com.darwinsys.util.jar .
```

The `jar` program and related tools add additional information to the manifest, including a listing of all the other files included in the archive.

The `java` command has a `-jar` option that tells it to run the main program found within a JAR file. In this case, it will also find classes it needs to load from within the same JAR file. How does it know which class to run? You must tell it. Create a one-line file like this, noting that the attribute fields are case-sensitive and that the colon must be followed by a space:

```
Main-Class: com.somedomainhere.HelloWorld
```

Place that in a file called, say, *manifest.stub*, and assume that you want to run the program `HelloWorld` from the given package. You can then use the following commands to package your app and run it from the JAR file:

```
C:> javac HelloWorld.java
C:> jar cvmf manifest.stub hello.jar HelloWorld.class
C:> java -jar hello.jar
Hello, World of Java
C:>
```

You can now copy the JAR file anywhere and run it the same way. You do not need to add it to your `CLASSPATH` or list the name of the main class.

On GUI platforms that support it, you can launch this application by double-clicking the JAR file. This works on macOS, Microsoft Windows, and most X Windows desktops.

In real life you would probably automate the JAR creation with Maven. Maven will automatically create a JAR file from your source project when you run `mvn package`. You can configure the appropriate Maven plug-in with `<manifest>` elements instead of having the information in a separate text file, as is done in the following:

```
<project ...>
  ...
  <packaging>jar</packaging>
  ...
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>2.4</version>
        <configuration>
          <archive>
            <manifest>
              <addclasspath>true</addclasspath>
              <mainClass>${main.class}</mainClass>
            </manifest>
          </archive>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

You have to provide the actual main class, of course, either as a `<property>` or by replacing `${main-class}` in the `<manifest>` section. With this in place, `mvn package` will build a runnable JAR file. However, if your class has external dependencies, the preceding steps will not package them, and you will get a missing class exception when you run it. For this, you need to use the Maven assembly plug-in:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-assembly-plugin</artifactId>
  <version>2.6</version>
  <configuration>
    <descriptorRefs>
      <descriptorRef>jar-with-dependencies</descriptorRef>
    </descriptorRefs>
    <archive>
      <manifest>
        <addDefaultImplementationEntries>true
        </addDefaultImplementationEntries>
        <mainClass>${main.class}</mainClass>
        <!-- <manifestFile>manifest.stub</manifestFile> -->
      </manifest>
      <manifestEntries>
        <Vendor-URL>https://YOURDOMAIN.com/SOME_PATH/</Vendor-URL>
      </manifestEntries>
    </archive>
  </configuration>
</plugin>
```

Now the invocation `mvn package assembly:single` will produce a runnable JAR with all dependencies. Note that your *target* folder will contain both *foo-0.0.1-SNAPSHOT.jar* and *foo-0.0.1-SNAPSHOT-jar-with-dependencies.jar*; the latter is the one you need.

The `jpackage` tool (see Recipe 2.18) will do a more complete job of packaging than `assembly:single` and is included with Java 14+.

# 1.10 Creating a JAR That Supports Multiple Versions of Java

## Problem

You are packaging a JAR file, whether a complete application or a library, for others to use. You want to use new Java features in code you release, but still provide backward compatibility for those who for some reason must run with older JDK releases.

## Solution

Create a multi-release JAR file.

## Discussion

"Always in motion the Java is," according to Yoda. This means devs always get cool new features to play with, or at least, to take advantage of. The multi-release JAR file provides a mechanism for forward compatibility to allow use of new versions of classes utilizing new features, while still allowing your code to run on older Java runtimes. To use this, provide multiple versions of the same source file updated and compiled for various releases. Create an extra directory in the JAR file for each newer-than-legacy version of the code. And you keep the older JDK installation around to compile the legacy version of the code.

This example uses a traditional Java `class` for pre-Java-16 JDKs, and a `record` for Java 16 and later. One requirement is that both must expose exactly the same API so that one can substitute for the other.

Here are the two versions of my trivial demo data object. They both have the same class name and the same list of methods (actually, only one method). Of course the source code must be in two different directories. First, the main program:

```
public class Main {
 public static void main(String[] args) {
  var type = new Data().type();
  System.out.printf("Run with a Data object of type '%s'.\n", type);
 }
}
```

Then there is a legacy (class) version of `Data`:

```
public class Data {
 String type() {
  return "class";
 }
}
```

Here is a Java 16 (record-based) version of the same `Data` class:

```
public record Data() {
 public String type() {
  return "record";
 }
}
```

It should be clear that either `Data` class will work with the main program. The `getType()` method is only there as a demo to identify which version is being called.

There is a `run_demo` script in the top-level diretory of `$js/multireleasejar`. The script runs the main program against both versions of `Data` without recompiling main, just to show that both work correctly and are interchangeable. The script then sets up the structure for the JAR file as follows:

```
|-- MANIFEST.MF
|-- META-INF
|  |-- MANIFEST.MF
|  +-- versions
|     +-- 16
|        +-- mrjdemo
|           +-- Data.class
+-- mrjdemo
      |-- Data.class
      +-- Main.class
```

The manifest file simply contains one line, `Multi-Release: true`, to inform Java to watch for multiple versions when classloading from the JAR.

Here is the `run_demo` script itself. Note that `withjava` is my custom command to run with a particular release of Java:

```
# run the multi-release-jar demo
D=/tmp/multirelease
mkdir $D
echo Running from directory with simple classpath
withjava 11 javac -d $D *.java
withjava 11 java -cp $D mrjdemo.Main
echo Running from directory with version up front in classpath
javac -d $D/META-INF/versions/16 versions/16/*.java
java -cp $D/META-INF/versions/16:$D mrjdemo.Main
cp MANIFEST.MF $D
cd $D
jar cvmf MANIFEST.MF /tmp/multirelease.jar mrjdemo META-INF
echo "Running from jar with legacy java"
withjava 11 java -cp /tmp/multirelease.jar mrjdemo.Main
echo "Running from jar with current java"
java -cp /tmp/multirelease.jar mrjdemo.Main
```

Running it produces this output:

```
$ sh run_demo
Running from directory with simple classpath
Run with a Data object of type 'class'.
Running from directory with version up front in classpath
Run with a Data object of type 'record'.
5 directories, 5 files
added manifest
adding: mrjdemo/(in = 0) (out= 0)(stored 0%)
adding: mrjdemo/Data.class(in = 275) (out= 215)(deflated 21%)
adding: mrjdemo/Main.class(in = 575) (out= 370)(deflated 35%)
ignoring entry META-INF/
adding: META-INF/versions/(in = 0) (out= 0)(stored 0%)
adding: META-INF/versions/16/(in = 0) (out= 0)(stored 0%)
adding: META-INF/versions/16/mrjdemo/(in = 0) (out= 0)(stored 0%)
adding: META-INF/versions/16/mrjdemo/Data.class(in = 1018) (out= 498)(deflated 51%)
Running from jar with legacy java
Run with a Data object of type 'class'.
Running from jar with current java
Run with a Data object of type 'record'.
$
```

The Java runtime correctly uses the `class` version when run on Java 11, and the `record` version when run on Java 16 or later.

The creation of the JAR file can be done a bit more automatically with some command-line options added to the `jar` command as of Java 9. It can also be automated using Maven or Gradle. I did it this "verbose" way to ensure that all the individual steps are visible.

# 1.11 Packaging Web Tier Components into a WAR File

## Problem

You have some web tier resources and want to package them into a single file for deploying to the server.

## Solution

Use `jar` to make a web archive (WAR) file. Or, as mentioned earlier, use Maven with `packaging="war"`.

## Discussion

Java EE/Jakarta EE defines a series of server-side components for use in web servers. They can be packaged for easy installation into a web server. A *web application* in the Servlet API specification is a collection of HTML/JSP/JSF pages, servlets, and other resources. A typical directory structure might include the following:

```
Project Root Directory
├── README.asciidoc
├── index.html - typical web pages
|── signup.jsp - ditto
└── WEB-INF Server directory
   ├── classes - Directory for individual .class files
   ├── lib    - Directory for Jar files needed by app
   └── web.xml - web app Descriptor ("Configuration file")
```

Once you have prepared the files in this way, you just package them up with a build tool. Using Maven, with `<packaging>war</packaging>`, your tree might look like this:

```
Project Root Directory
├── README.asciidoc
├── pom.xml
└── src
   └── main
      ├── java
      │   └── foo
      │      └── WebTierClass.java
      └── webapp
         ├── WEB-INF
         │   ├── classes
         │   ├── lib
         │   └── web.xml
         ├── index.html
         └── signup.jsp
```

Then `mvn package` will compile things, put them in place, and create the WAR file for you, leaving it under *target*. Gradle users would use a similar directory structure. You then deploy the resulting WAR file into your web server. For details on the deployment step, consult the documentation on the particular server you're using.

# 1.12 Compiling and Running Java: GraalVM for Better Performance

## Problem

You've heard that Graal is a JVM from Oracle that's faster than the standard JDK, and you want to try it out. Graal promises to offer better performance, largely obtained by pre-compiling your Java code into executable form for a given platform. And it offers the ability to mix and match programming languages.

## Solution

Download and install GraalVM.

## Discussion

GraalVM bills itself as "a universal virtual machine for running applications written in JavaScript, Python, Ruby, R, JVM-based languages like Java, Scala, Clojure, Kotlin, and LLVM-based languages such as C and C++."

Note that Graal is undergoing change. There may be newer versions and changed functionality by the time you are ready to install.

You could build your own GraalVM, since the complete source is on GitHub. However, it's easier to download a pre-built binary. Start at the downloads page. You can download a tarball for Linux, macOS, and Windows. There is no formal installer at this point. To install it, open a terminal window and try the following (the directory chosen is for macOS):

```
$ cd /Library/Java/JavaVirtualMachines # or ~/Library/...
# In this, replace 23, macos and aarch64 with your actual download:
$ sudo tar xzvf ~/Downloads/graalvm-jdk-23_macos-aarch64_bin.tar.gz
$ cd
$ /usr/libexec/java_home -V # macOS only
    24 (arm64) "Oracle Corporation" - "OpenJDK 24-ea"
        /Library/Java/JavaVirtualMachines/jdk-24.jdk/Contents/Home
    23.0.1 (arm64) "Oracle Corporation" - "Oracle GraalVM 23.0.1+11.1"
        /Library/Java/JavaVirtualMachines/graalvm-jdk-23.0.1+11.1/Contents/Home
    23 (arm64) "Oracle Corporation" - "OpenJDK 23-ea"
        /Library/Java/JavaVirtualMachines/jdk-23.jdk/Contents/Home
    22 (arm64) "Oracle Corporation" - "OpenJDK 22-ea"
        /Library/Java/JavaVirtualMachines/jdk-22.jdk/Contents/Home
    17.0.11 (arm64) "JetBrains s.r.o." - "JBR-17.0.11+1-1312.2-nomod 17.0.11"
        /Users/ian/Library/Java/JavaVirtualMachines/jbr-17.0.11/Contents/Home
    21.0.1 (arm64) "Eclipse Adoptium" - "OpenJDK 21.0.1"
        /Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home
$
```

macOS does not provide a means to permanently set the default version; you have to adjust `JAVA_HOME` each time you want to change. The `java_home` command output confirms that you have installed GraalVM, but it's not your default JVM yet:

```
export JAVA_HOME=$(/usr/libexec/java_home -v 23.0.1)
```

On most versions of Linux, after installing a JDK, you can use the standard Linux `alternatives` command (e.g., `sudo alternatives --config java`) to make this your default. On other systems, do the install in a sensible place, and set `JAVA_HOME` and/or `PATH` as needed.

Now you should be running the Graal version of Java. This is what you should see:

```
$ java -version
java version "23.0.1" 2024-10-15
Java(TM) SE Runtime Environment Oracle GraalVM 23.0.1+11.1
    (build 23.0.1+11-jvmci-b01)
Java HotSpot(TM) 64-Bit Server VM Oracle GraalVM 23.0.1+11.1
    (build 23.0.1+11-jvmci-b01, mixed mode, sharing)
$
```

Your output will probably differ, but as long as it looks vaguely like this and says "GraalVM" you should be good.

Graal includes a number of useful tools, including `native-image`, which can in some cases translate a class file into a binary executable for the platform it's running on, optimizing startup speed and also reducing the download size needed to run a single appli-

cation. The `native-image` tool is included in the download. This tool reads the compiled *.class* file, so if you have set your `PATH` correctly, then:

```
$ javac Hello.java
$ native-image Hello
```

should produce a runnable binary. Here's an example. The first command is just to show that Graal generally works like a JDK VM and that it can compile and run a Java program. Then we build and run a native-compiled version:

```
$ java Hello.java
Hello on 2024-10-24
$ javac Hello.java
$ native-image Hello
========================================================================================
GraalVM Native Image: Generating hello (executable)...
========================================================================================
[1/8] Initializing...                          (23.1s @ 0.07GB)
[2/8] Performing analysis...  []               (3.5s @ 0.24GB)
[3/8] Building universe...                     (0.6s @ 0.23GB)
[4/8] Parsing methods...      []               (0.4s @ 0.24GB)
[5/8] Inlining methods...     []               (0.4s @ 0.26GB)
[6/8] Compiling methods...    []               (7.5s @ 0.23GB)
[7/8] Laying out methods...   []               (0.5s @ 0.27GB)
[8/8] Creating image...       []               (0.9s @ 0.23GB)
----------------------------------------------------------------------------------------
Build artifacts:
 /tmp/hello (executable)
========================================================================================
Finished generating hello in 37.3s.
$ ./hello
Hello on 2024-10-24
$
```

The `native-image` tool does take longer to create its output, and it is *extremely* verbose, both compared to `javac` (about 30 lines had to be elided from the preceding printout to keep it short enough). There is an option, `--silent`, to make `native-image` behave like `javac` in this regard, but it's worth running without this option the first time. After that, you may wish to `alias native-image="native-image --silent"` (or write a batch file or script if you're using Windows `cmd` or PowerShell).

You can also use Graal to mix-and-match languages like Python and JavaScript. We'll explore calling some other languages from Java in Recipe 18.4.

See the GraalVM website for more information on GraalVM.

# 1.13 Getting Information About the Environment, OS, and Runtime

## Problem

You need to know how your Java program can deal with its immediate surroundings with what we call the runtime environment. In one sense, everything you do in a Java program using almost any Java API involves the environment. This recipe covers environment variables, system properties, and operating-system dependencies.

## Solution

Use methods of the `System` and `Runtime` classes, which know a lot about your particular system.

## Discussion

There are several locations from which to obtain this information.

### Getting environment variables

Use `System.getenv()`.

The seventh edition of Unix, released in 1979, had a then-new feature known as environment variables. Environment variables are in all modern Unix systems (including macOS) and in most later command-line systems, such as the DOS or Command Prompt in Windows. Environment variables are commonly used for customizing an individual computer user's runtime environment, hence the name. To take one familiar example, on Unix or DOS the environment variable `PATH` determines where the system looks for executable programs. So, of course developers want to know how to access environment variables from their Java program.

The answer is that you can do this in all modern versions of Java. In some ancient versions of Java, `System.getenv()` was deprecated and/or just didn't work. Nowadays the `getenv()` method is no longer deprecated, though it still carries the warning that system properties (see "Getting information from system properties") should be used instead. Even among systems that support environment variables, their names are case-sensitive on some platforms and case-insensitive on others. The code in Example 1-6 is a short program that uses the `getenv()` method. Note that it is spelled `getenv()`, not the expected `getEnv()`—and it's probably too old to change.

**Example 1-6. main/src/main/java/environ/GetEnv.java**

```
public class GetEnv {
  public static void main(String[] argv) {
    String envVarName = argv.length == 0 ? "PATH" : argv[0];
    System.out.printf("System.getenv(\"%s\") = %s\n",
      envVarName, System.getenv(envVarName));
  }
}
```

Running this code on Windows will produce output similar to the following, with your actual `PATH` instead of one I had a while ago:

```
System.getenv("PATH") = C:\windows\bin;c:\jdk-17\bin;c:\Users\ian\bin
```

On Unix (where most nonsystem programs such as Java are installed in */usr/local/bin*), the code is:

```
System.getenv("PATH") = /home/ian/bin:/bin:/usr/bin:/sbin:/usr/sbin:/usr/local/bin
```

The no-argument form of the method `System.getenv()` returns *all* the environment variables in the form of an immutable `StringMap`. You can iterate through this map and access all the user's settings or retrieve multiple environment settings.

### Getting information from system properties

To get information from the system properties, use `System.getProperty()` or `System.getProperties()`.

What is a *property* anyway? A property is just a name and value pair stored in a `java.util.Properties` object, which we discuss more fully in Recipe 7.10.

The `System.Properties` object controls and describes the Java runtime. The `System` class has a static `Properties` member whose content is the merger of operating system specifics (`os.name`, for example), system and user tailoring (`java.class.path`), and properties defined on the command line (as we'll see in a moment). Note that the use of periods in the names of system properties (like `os.arch`, `os.version`, `java.class.path`, and `java.lang.version`) makes it look as though there is a hierarchical relationship similar to that for package/class names. The `Properties` class, however, imposes no such relationships: each key is just a string, and dots are not special.

To view all the defined system properties, you can iterate through the output of calling `System.getProperties()` as in Example 1-7.

**Example 1-7.** jshell System.getProperties()

```
jshell> System.getProperties().forEach((k,v) -> System.out.println(k + "->" +v))
awt.toolkit->sun.awt.X11.XToolkit
java.specification.version->21
sun.cpu.isalist->
sun.jnu.encoding->UTF-8
java.class.path->.
java.vm.vendor->Oracle Corporation
sun.arch.data.model->64
java.vendor.url->https://java.oracle.com/
user.timezone->
os.name->OpenBSD
java.vm.specification.version->21
... many more ...
jshell>
```

Note that properties whose names begin with `sun` are unsupported and subject to change. There used to be a variable named `sun.boot.class.path`, which contained a list of the archives used at JVM startup. But that entry is not found anymore. Let's look for any property with `boot` in its name:

```
jshell> System.getProperties().forEach((k,v) -> {
 ... if (((String)k).contains("boot")) System.out.println(k + "->" +v);})
sun.boot.library.path->/usr/local/jdk-21/lib
```

So a boot library path is there (the location will vary on different OSes), but no other `boot` entries. As stated, "subject to change."

To retrieve one system-provided property, use `System.getProperty(propName)`. If I wanted to find out if the system properties had a property named `"pencil_color"`, I could say:

```
    String sysColor = System.getProperty("pencil_color");
```

But what does that return? Surely Java isn't clever enough to know about everybody's favorite pencil color? Right you are! But we can easily tell Java about our pencil color (or anything else we want to tell it) using the `-D` argument.

When starting a Java command, define a value in the `System.Properties` object just by using a `-D` option. Its argument must have a name, an equals sign, and a value, which are parsed the same way as in a properties file (see Recipe 7.10). You can have more than one `-D` definition between the `java` command and your class name on the command line. At the Unix or Windows command line, type:

```
$ java -D"pencil_color=Deep Sea Green" environ.SysPropDemo
```

Here's the result:

```
-- listing properties --
java.specification.version=21
jdk.internal.javac.source=21
sun.jnu.encoding=UTF-8
java.class.path=
pencil_color=Deep Sea Green
java.vm.vendor=Oracle Corporation
... many more ...
```

When running this under an IDE, put the variable's name and value in the appropriate dialog box, for example, in Eclipse's Run Configuration dialog under Program Arguments. You can also set environment variables and system properties using the build tools (Maven, Gradle, etc.).

The `SysPropDemo` program has code to extract just one or a few properties, so you can run it like this:

```
$ java environ.SysPropDemo os.arch
os.arch = x86
```

If you invoke the `SysPropDemo` program with no arguments, it outputs the same information as the `jshell` fragment in Example 1-7.

Which reminds me—this is a good time to mention system-dependent code. "Dealing with code that depends on the Java version or the operating system" talks about OS-dependent code and release-dependent code.

Recipe 7.10 lists more details on using your own `Properties` files. The Javadoc page for `java.util.Properties` lists the exact rules used in the `load()` method, as well as other details.

### Dealing with code that depends on the Java version or the operating system

In the very rare case when you need to write code that adapts to the underlying operating system, use `System.Properties` to find out the Java version and the operating system, or use the `Files` and `Path` classes to check whether certain system-dependent files or directories are present. You can also use `java.awt.TaskBar` to see if you can use the system-dependent taskbar or dock, and `java.awt.Desktop` to see if you can open a file in the default browser, for example.

Some things depend on the version of Java you are running. Use `System.getProperty()` with an argument of `java.specification.version` to find out the Java version.

Alternatively, and with greater generality, you may want to test for the presence or absence of particular classes. One way to do this is with `Class.forName("class")` (see Chapter 17), which throws an exception if the class cannot be loaded—a good indication that it's not present in the runtime's library. Example 1-8 shows code for this, from an application wanting to find out whether the common Swing UI components are available. The Javadoc for the standard classes reports the version of the JDK in which this class first appeared, under the heading "Since." If there is no such heading, it normally means that the class has been present since the beginnings of Java.

Another alternative for dealing with different Java versions is the multi-release JAR file, covered in Recipe 1.10.

**Example 1-8.** ***main/src/main/java/starting/CheckForSwing.java***

```
public class CheckForSwing {
  public static void main(String[] args) {
    try {
      Class.forName("javax.swing.JButton");
    } catch (ClassNotFoundException e) {
      String failure =
        "Sorry, but this version of MyApp needs \n" +
        "a Java Runtime with javax.swing GUI components.\n" +
        "Please check your Java installation and try again.";
      // Better to make something appear in the GUI. Either a
      // Dialog, or: myPanel.add(new Label(failure));
      // Can't be JDialog as Swing isn't present.
      System.err.println(failure);
    }
    // No need to print anything here - the GUI should work...
  }
}
```

It's important to distinguish between testing this code at compile time and at runtime. In both cases, it must be compiled on a system that includes the classes you are testing for: in this case, JDK >= 1.1 and Swing, respectively. These tests are only attempts to help the poor backwater Java runtime user trying to run your up-to-date application. The goal is to provide this user with a message more meaningful than the simple "class not found" error that the runtime gives. Put the test early in the main flow of your application, before any GUI objects are constructed. Otherwise the code just sits there wasting space on newer runtimes and never gets run on Java systems that don't include Swing. Obviously this is a very early example, but you can use the same technique to test for any

runtime feature added at any stage of Java's evolution (see Appendix A for an outline of the features added in each release of Java). You can also use this technique to determine whether a needed third-party library has been successfully added to your `CLASSPATH`. You can also use the Reflection API to determine if a class has a given method; see Chapter 17.

Also, although Java is designed to be portable, some things aren't. These include such variables as the filename separator. Everybody on Unix knows that the filename separator is a slash character (/) and that a backward slash, or backslash (\), is an escape character. Back in the late 1970s, a group at Microsoft was actually working on Unix—their version was called Xenix, later taken over by SCO—and the people working on DOS saw and liked the Unix filesystem model. The earliest versions of MS-DOS didn't have directories; it just had user numbers like the system it was a quick and dirty clone of, Digital Research CP/M (itself using ideas from various other systems). So the Microsoft developers set out to clone the Unix filesystem organization. Unfortunately, MS-DOS had already committed the real slash character for use as an option delimiter, for which Unix had used a dash (`-`); and the `PATH` separator (:) was also used as a drive letter delimiter, as in `C:` or `A:`. So we now have commands like those shown in Table 1-6.[4]

Table 1-6. Directory listing commands

| System | Directory list command | Meaning | Example PATH setting |
|---|---|---|---|
| Unix | `ls -R /` | Recursive listing of /, the top-level directory | `PATH=/bin:/usr/bin` |
| DOS | dir /s \ | Directory with subdirectories option (i.e., recursive) of \, the top-level directory (but only of the current drive) | PATH=C:\windows;D:\mybin |

Where does this get us? If we are going to generate filenames in Java, we may need to know whether to put a / or a \ or some other character. Java has two solutions to this. First, when moving between Unix and Microsoft systems, at least, it is *permissive*: either / or \ can be used,[5] and the code that deals with the operating system sorts it out. Second, and more generally, Java makes the platform-specific information available in a platform-independent way. For the file separator (and also the `PATH` separator), the `java.io.File` class makes available some static variables containing this information. Because the `File` class manages platform-dependent information, it makes sense to anchor this information here. The variables are shown in Table 1-7.

Table 1-7. Filename properties

| Name | Type | Meaning |
|---|---|---|
| `separator` | `static String` | The system-dependent filename separator character (e.g., / or \), as a string for convenience |
| separatorChar | static char | The system-dependent filename separator character (e.g., / or \) |
| `pathSeparator` | `static String` | The system-dependent path separator character, as a string for convenience |
| pathSeparatorChar | static char | The system-dependent path separator character |

Both filename and path separators are normally characters, but they are also available in `String` form for convenience.

You can use the `System.properties` object to determine the operating system you are running on. We previously showed some code that simply lists the system properties; it can be informative to run that on several different Java implementations on different OSes.

Some OSes, for example, provide a mechanism called the null device that can be used to discard output (typically used for timing purposes, or to discard standard output while retaining system error output). Here is code that asks the system properties for the `os.name` and uses it to make up a name that can be used for discarding data (if no null device is known for the given platform, we return the name `jnk`, which means that on such platforms, we'll occasionally create, well, junk files; I just remove these files when I stumble across them):

```
package com.darwinsys.lang;

import java.io.File;

/** Some things that are system dependent.
 * All methods are static.
 * @author Ian Darwin
 */
public class SysDep {
```

```
    final static String UNIX_NULL_DEV = "/dev/null";
    final static String WINDOWS_NULL_DEV = "NUL:";
    final static String FAKE_NULL_DEV = "jnk";

    /** Return the name of the "Null Device" on platforms which support it,
     * or "jnk" (to create an obviously well-named temp file) otherwise.
    * @return The name to use for output.
     */
    public static String getDevNull() {

        if (new File(UNIX_NULL_DEV).exists()) {    ❶
            return UNIX_NULL_DEV;
        }

        String sys = System.getProperty("os.name"); ❷
        if (sys==null) {                            ❸
            return FAKE_NULL_DEV;
        }
        if (sys.startsWith("Windows")) {            ❹
            return WINDOWS_NULL_DEV;
        }
        return FAKE_NULL_DEV;                       ❺
    }
}
```

❶ If `/dev/null` exists, use it.

❷ If not, ask `System properties` if it knows the OS name.

❸ Nope, so give up, return `jnk`.

❹ We know it's Microsoft Windows, so use `NUL:`.

❺ All else fails, go with `jnk`.

1 A JEP is a Java Enhancement Proposal, part of the OpenJDK project's processes for "moving Java forward." There's a discussion at *https://www.infoworld.com/article/2335544*.

2 Git is not an acronym. The name was chosen by Linux founder Linus Torvalds; he once quipped that he had named the software after himself. Due to Git's popularity, there is already one fairly complete clone called Got, which stands for "Game of Trees", a pun involving source code directory trees and a once-popular TV show.

3 Some people like to use names like *MyPackage.mf* so that it's clear which package it is for; the extension *.mf* is arbitrary, but it's a good convention for identifying manifest files.

4 People only exposed to MS-Windows tend to sometimes refer to the character \ as "slash" instead of "backslash" or to write URLs with \ instead of /. Ah well.

5 When compiling strings with backslashes for use on Windows, remember to double them because \ is an escape character in most places other than the MS-DOS command line: `String rootDir = "C:\\";`.

Previous chapter
< Preface

Next chapter
2. Software Development, Testing, and Maintenance >